



Cairo University

Faculty of Computers and Artificial Intelligence

CS251

Introduction to Software Engineering

Budgeting app (Masroofy)

Software Design Specifications v1.0

S26

ID	Name
20242447	Yehia Hassan Abdelmoaty
20240794	Noha Mohamed Ahmed
20240759	Jana Ahmed Farhat Hassan
20240650	Hana Khaled Abdelhamed

April 2026



CS251 Project: **Masroofy Budgeting App**

Software Design Specification

Contents

Team.....	3
Document Purpose and Audience.....	3
System Models.....	4
I. Architecture Diagram	7
II. Class Diagram(s).....	9
III. Class Descriptions.....	10
IV. Sequence diagrams.....	11
Class - Sequence Usage Table.....	15
V. State Diagram.....	16
VI. SOLID Principles.....	17
VII. Design Patterns.....	18
Tools.....	19
Ownership Report.....	19



CS251 Project: Masroofy Budgeting App

Software Design Specification

Team

ID	Name	Email	Mobile
20242447	Yehia Hassan Abdelmoaty	20242447@stud.fci-cu.edu.eg	01205309575
20240794	Noha Mohamed Ahmed	20240794@stud.fci-cu.edu.eg	01007021975
20240759	Jana Ahmed Farahat Hassan	20240759@stud.fci-cu.edu.eg	01281727773
20240650	Hana Khaled Abdelhamed	20240650@stud.fci-cu.edu.eg	01060820155

Document Purpose and Audience

Document Purpose

This Software Design Specification (SDS) provides the technical blueprint for the budgeting application. It translates the functional requirements into concrete architectural and structural designs, defining the specific principles and design patterns required to build a scalable and maintainable system.

Target Audience

This document is intended for the project's participants, including the software development team, software architects, quality assurance testers, and project managers. It serves as a definitive guide for coding, validating system architecture, developing test cases, and verifying that the technical design strictly maps to the established project scope.



CS251 Project: Masroofy Budgeting App

Software Design Specification

System Models

I. Architecture Diagram

1. System Components and Subsystems

The system is structured using a layered approach based on the MVC pattern. Each component has a clear responsibility to ensure separation of concerns and maintainability.

1. User (Client)

The user represents the end actor who interacts with the system through a web or mobile interface. The user performs actions such as adding expenses, viewing history, and managing budget data.

2. View Layer (Presentation Layer)

The View layer is responsible for the user interface and user experience.

Responsibilities:

- Displaying data to the user
- Collecting user input
- Rendering screens such as:
 - Dashboard
 - Add Expense
 - History
 - Settings

This layer does not contain business logic; it only communicates with the Controller.



CS251 Project: Masroofy Budgeting App

Software Design Specification

3. Controller Layer (Application Logic)

The Controller layer acts as the intermediary between the View and the Model.

Responsibilities:

- Handling user requests from the View
- Validating input data
- Applying business rules
- Calling the appropriate Model functions
- Returning results back to the View

Examples of controllers:

- Expense Controller (Add / Edit / Delete)
- Budget Controller
- Cycle/Management Controller

4. Model Layer (Data & Business Logic)

The Model layer represents the core data and business rules of the system.

Responsibilities:

- Managing application data
- Defining entities and their relationships
- Applying core business logic

Examples of models:

- Expense Model
- Budget Model
- Category Model

The Model communicates with the Database to store and retrieve data.



CS251 Project: Masroofy Budgeting App

Software Design Specification

5. Database Layer

The Database layer is responsible for persistent data storage.

Responsibilities:

- Storing all system data
- Providing data retrieval and update operations

Example tables:

- Expenses Table
 - Budgets Table
 - Categories Table
-

2. Architectural Design

The system follows the **Model–View–Controller (MVC)** architectural pattern.

Why MVC was chosen:

- Ensures **separation of concerns**
 - Improves **code maintainability**
 - Allows **independent development** of UI and logic
 - Enhances **scalability and testing**
-

MVC Structure Explanation

- **Model:**
Handles data and business logic.
- **View:**
Displays data and interacts with the user.
- **Controller:**
Processes user input and coordinates between Model and View.

CU – FCAI – CS251 Introduction to Software Engineering – 2026 - Software Design Specifications

Prepared by Mostafa Saad and Mohammad El-Ramly V1.0

Edited by Mohamed Samir, Updated to V2.0 by Mohammad El-Ramly 10/4/2020 and V3.0 25/5/2021, 20/4/2025



CS251 Project: Masroofy Budgeting App

Software Design Specification

How the system works (Flow):

1. The **User** interacts with the **View (UI)**
 2. The View sends the request to the **Controller**
 3. The Controller processes the request and calls the **Model**
 4. The Model interacts with the **Database**
 5. The result is returned back:
 - Model → Controller → View → User
-

3. Architectural Diagram

The system architecture diagram is designed using a **layered MVC structure**.

It includes the following components:

- User
- View Layer (UI)
- Controller Layer
- Model Layer
- Database Layer

Diagram Description

- The **User** communicates with the **View layer**
- The **View** sends requests to the **Controller**
- The **Controller** interacts with the **Model**
- The **Model** communicates with the **Database**
- Data flows back in the reverse direction to be displayed to the user

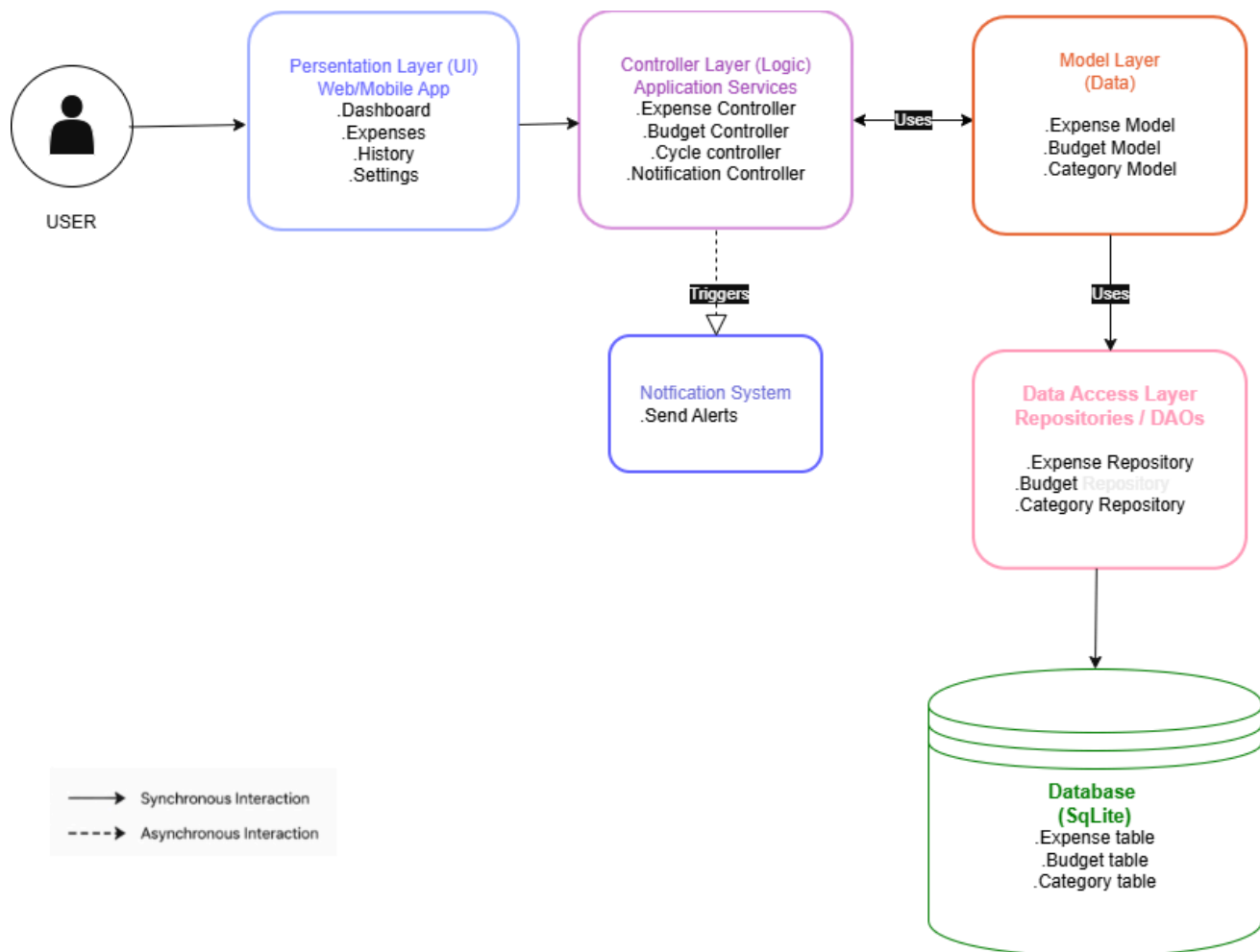
The diagram uses:

- **Boxes** to represent system components
- **Arrows** to represent data flow and interaction
- A **layered structure** to clearly show separation of responsibilities



CS251 Project: Masroofy Budgeting App

Software Design Specification

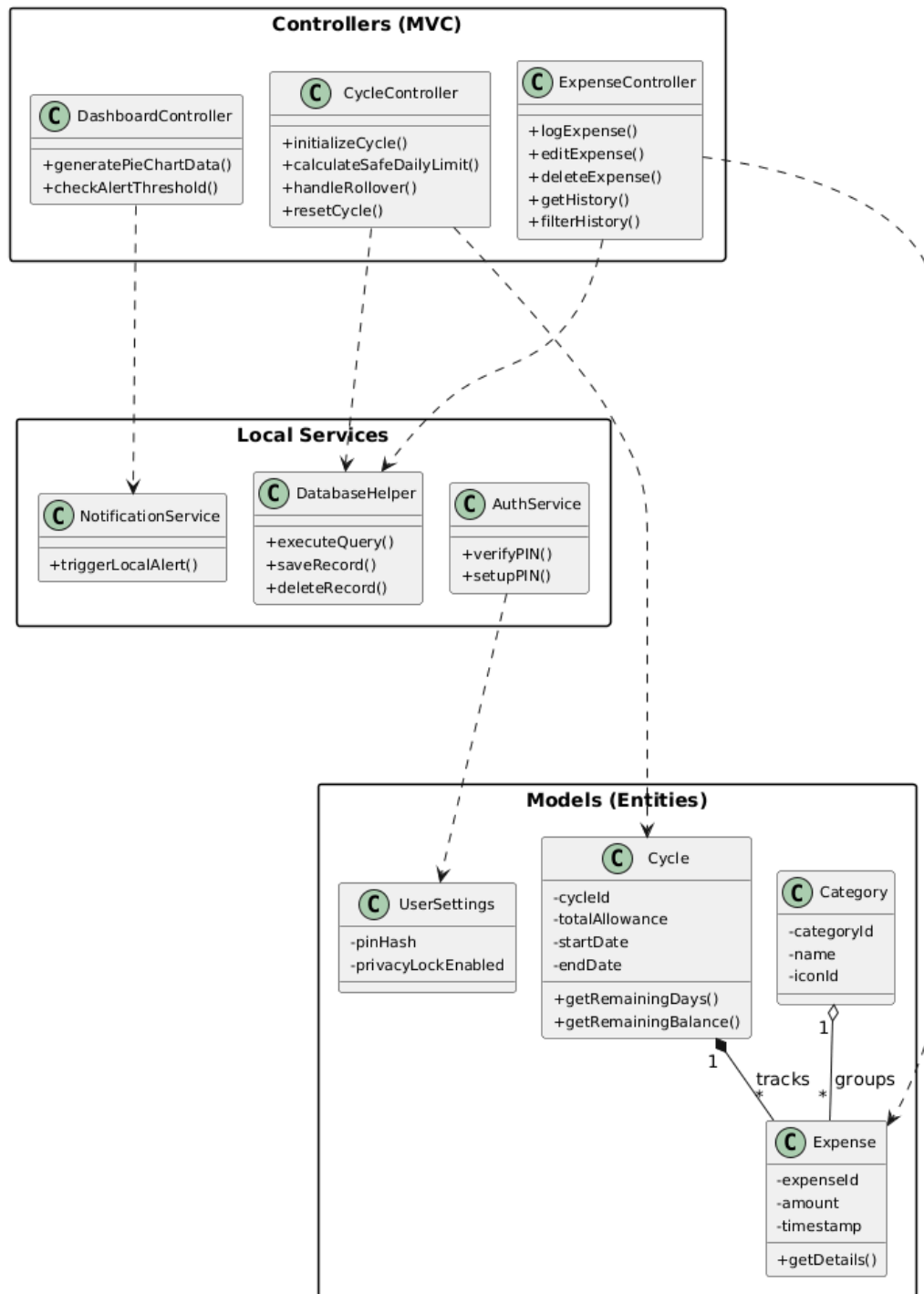




CS251 Project: Masroofy Budgeting App

Software Design Specification

II. Class Diagram(s)





CS251 Project: Masroofy Budgeting App

Software Design Specification

III. Class Descriptions

Class ID	Class Name	Description & Responsibility
1.	Cycle	Represents the user's budget cycle. Tracks the total cash allowance, start/end dates, and calculates remaining days.
2.	Expense	Represents a logged financial transaction. Stores the numerical amount, timestamp, and linked category ID.
3.	Category	Represents a lifestyle spending group (e.g., Food, Transport) used to classify expenses.
4.	UserSettings	Stores the user's hashed 4-digit PIN and the boolean flag for the privacy lock.
5.	CycleController	Manages the core budget timeframe logic. Responsible for initializing allowances, calculating the "Safe Daily Limit", managing midnight rollovers, and resetting cycles.
6.	ExpenseController	Handles the logic for rapid manual expense logging, filtering transaction history by date/category, and editing/deleting past entries.
7.	DashboardController	Processes mathematical data for the UI view. Generates pie chart percentages for visual spending insights and monitors the 80% usage threshold.
8.	DatabaseHelper	Manages the local SQLite database engine to persist cycles and transactions securely on the device without internet access.
9.	NotificationService	Triggers local device push notifications when budget thresholds are breached to warn the user.
10.	AuthService	Authenticates the user via the 4-digit PIN or biometric lock before granting application access.



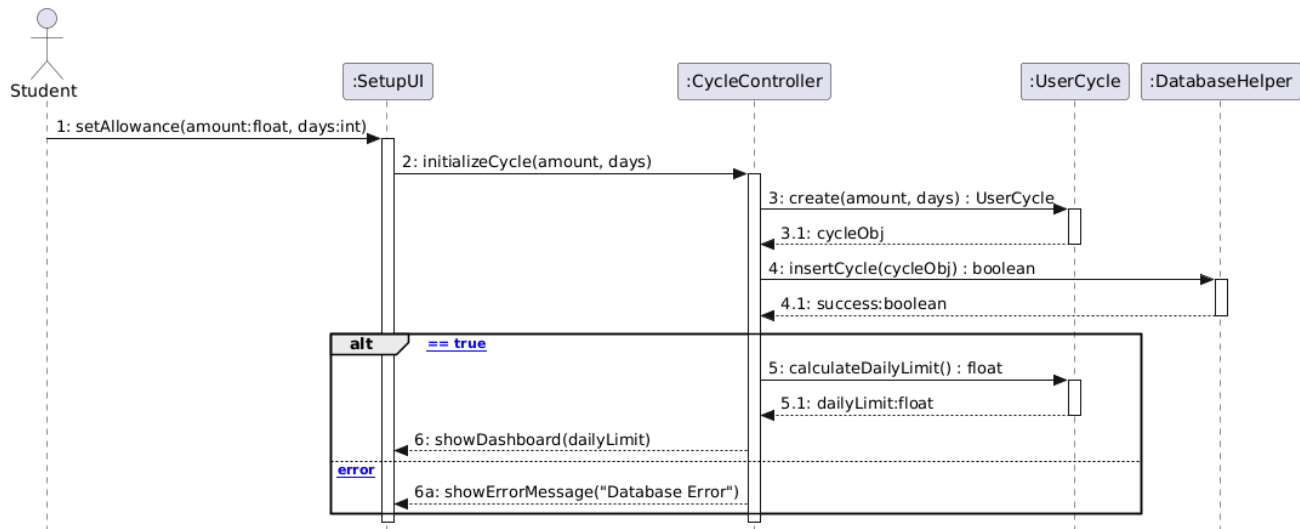
CS251 Project: Masroofy Budgeting App

Software Design Specification

IV. Sequence diagrams

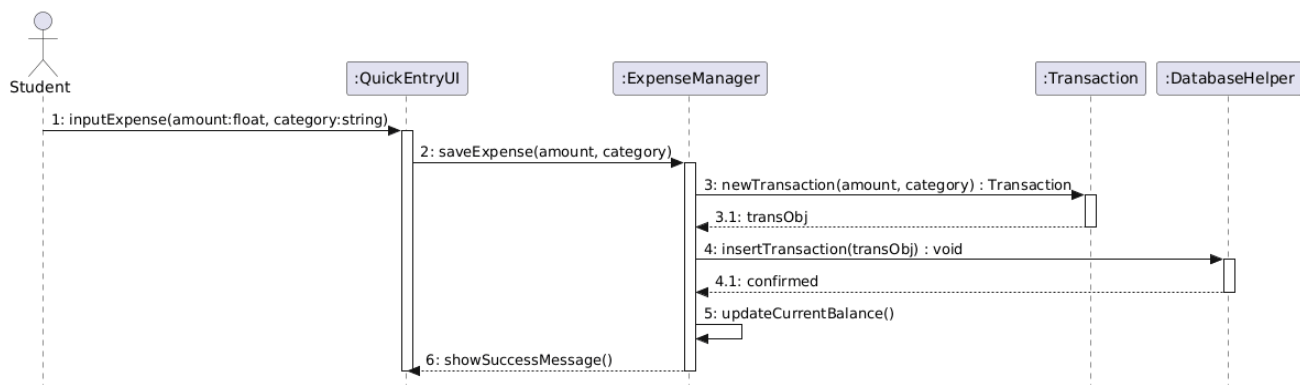
1. Set Initial Budget Cycle (US #1)

This matches SRS requirement for setting the 30-day allowance.



2. Log Daily Expense (US #2)

This covers the "Rapid Entry" requirement (Quick-add).



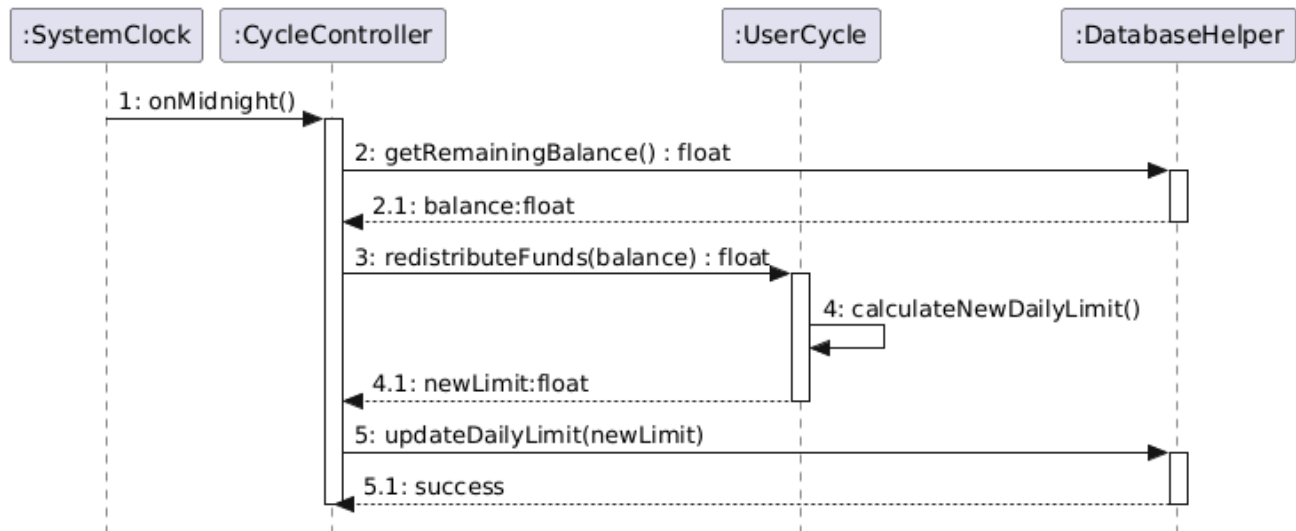


CS251 Project: Masroofy Budgeting App

Software Design Specification

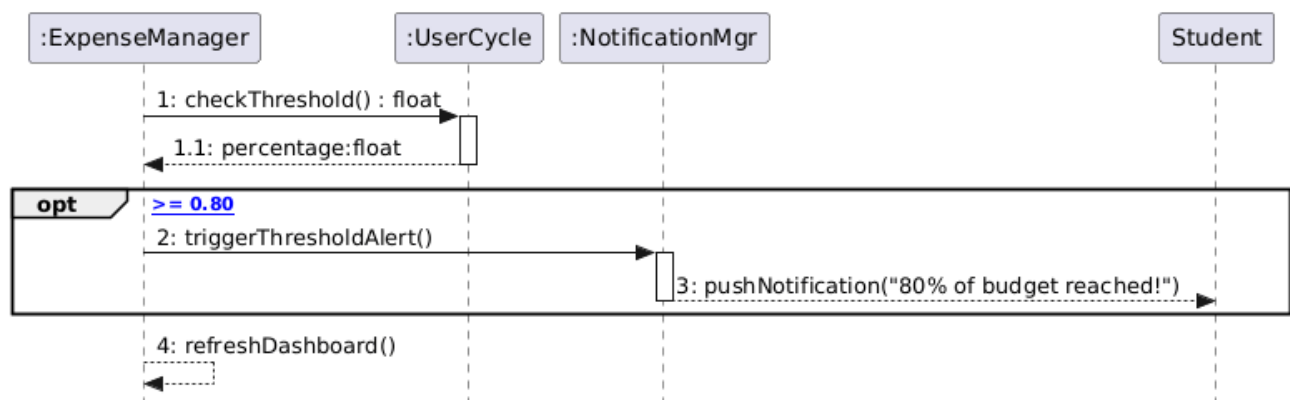
3. Automatic Midnight Rollover (US #4 / FR-4)

This shows the system automatically recalculating the limit.



4. Threshold Alert (80% Spending)

This shows the logic for triggering the notification alert.



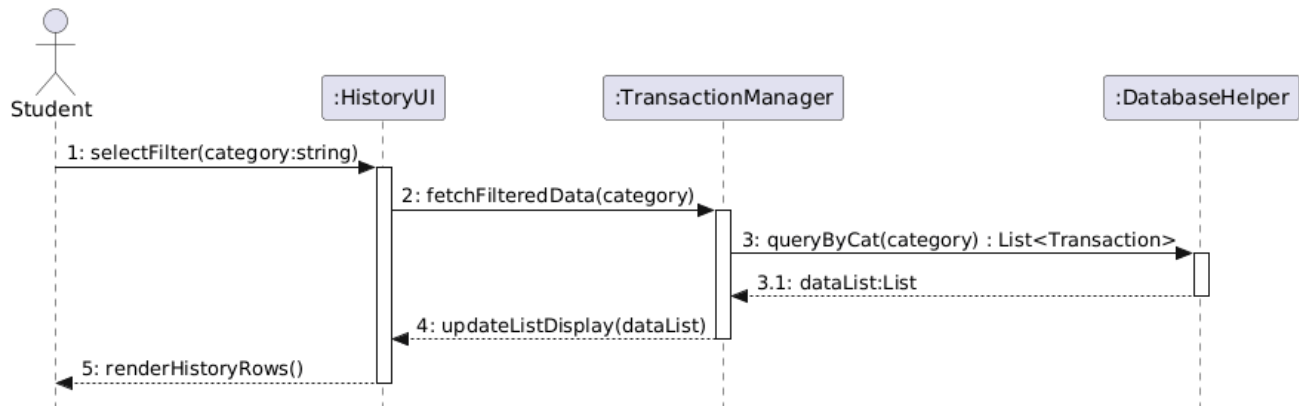


CS251 Project: Masroofy Budgeting App

Software Design Specification

5. View History & Filters

Based on the Data Dictionary entry for "Filter Selection."



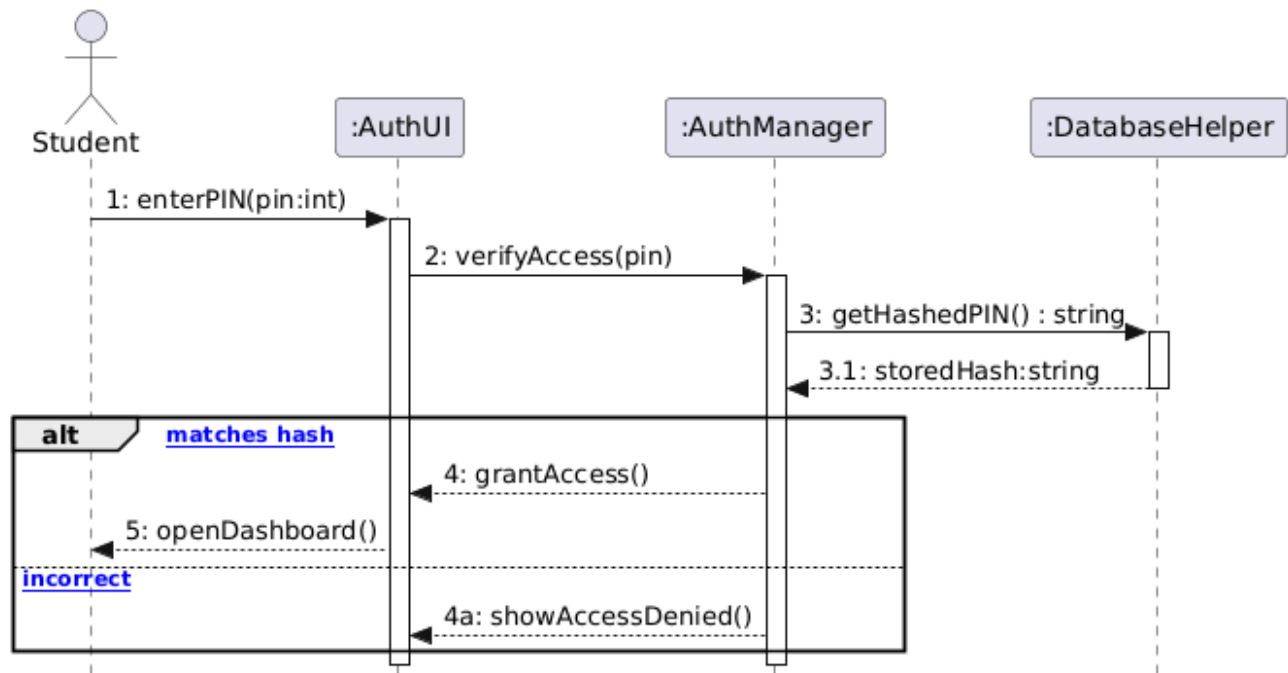


CS251 Project: Masroofy Budgeting App

Software Design Specification

6. Set/Verify Privacy PIN (US #11)

Matches "LocalAuth" class and "PIN Field" data entry.





CS251 Project: Masroofy Budgeting App

Software Design Specification

Class - Sequence Usage Table

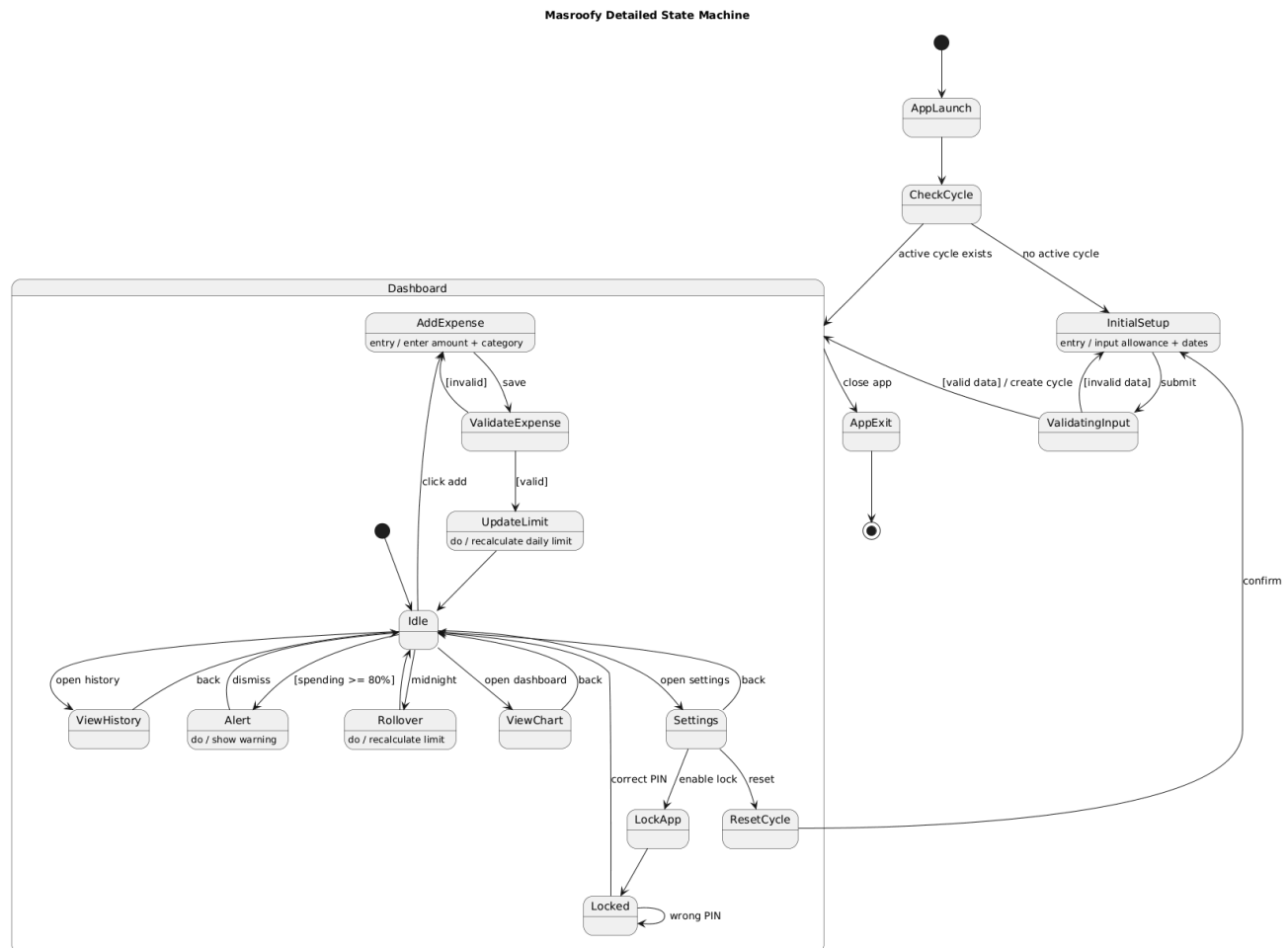
Sequence Diagram	Classes Used	All Methods Used
1.Set Initial Cycle	CycleController, UserCycle, DatabaseHelper	initializeCycle(), createNewCycle(), calculateDailyLimit() .
2. Log Expense	ExpenseManager, Transaction, DatabaseHelper	saveExpense(), newTransaction(), insertTransaction()
3.Midnight Rollover	CycleController, UserCycle, DatabaseHelper	onMidnight(), redistributeFunds(), updateDailyLimit()
4. Threshold Alert	ExpenseManager, UserCycle, NotificationMgr	checkThreshold(), triggerThresholdAlert(), pushNotification())
5.View History	TransactionManager, DatabaseHelper	fetchFilteredData(), queryByCat(), updateListDisplay()
6.Privacy PIN	AuthManager, DatabaseHelper	verifyAccess(), getHashedPIN(), grantAccess()



CS251 Project: Masroofy Budgeting App

Software Design Specification

V. State Diagram





CS251 Project: **Masroofy Budgeting App**

Software Design Specification

VI. SOLID Principles

1. Single Responsibility Principle (SRP)

- **Where it was applied:** The separation between the ExpenseController and the DatabaseHelper.
- **Explanation:** According to SRP, a class should have only one reason to change. In our design, the ExpenseController handles only the business logic (validating expense amounts, categorizing). It delegates all data persistence to the DatabaseHelper. If the SQLite query syntax changes, only the DatabaseHelper is modified; if the logic for calculating remaining balances changes, only the Controller is modified.

2. Open/Closed Principle (OCP)

- **Where it was applied:** The AuthService and system security checks.
- **Explanation:** The software is open for extension but closed for modification. Currently, Masroofy uses a 4-digit PIN for the privacy lock. The AuthService is designed so that if we want to introduce Biometric (Fingerprint/FaceID) locks in future updates, we can extend the authentication modules without modifying or breaking the existing PIN-checking codebase.

3. Dependency Inversion Principle (DIP)

- **Where it was applied:** The relationship between high-level controllers (e.g., CycleController) and data access (DatabaseHelper).
- **Explanation:** High-level modules should not depend on low-level modules. Instead of the controllers executing raw SQLite queries directly (which tightly couples them to a specific database engine), they depend on the abstract methods provided by DatabaseHelper (like saveRecord()). This abstraction makes the system easier to test and maintain.



CS251 Project: Masroofy Budgeting App

Software Design Specification

VII. Design Patterns

1. Singleton Pattern

- **Where you used it:** The DatabaseHelper class.
- **Benefit of using it:** It ensures the application only ever creates one single connection to the local SQLite database. This prevents database file-locking crashes and memory leaks on the user's mobile device.

2. Observer Pattern

- **Where you used it:** The interaction between the ExpenseController, DashboardController, and NotificationService.
- **Benefit of using it:** It allows parts of the app to automatically react to changes. When a user adds a new expense, the dashboard chart immediately updates and the system checks if a warning notification is needed, all without the controllers being tightly linked together.

3. Facade Pattern

- **Where you used it:** The DatabaseHelper class acting as an interface for the Controllers.
- **Benefit of using it:** It hides the complex, low-level code required to manage native SQLite database connections. Instead of dealing with complex database cursors and queries, the rest of the app just uses simple, clean methods like saveRecord().



CS251 Project: Masroofy Budgeting App

Software Design Specification

Tools

- <https://www.plantuml.com/> for Class, State, and Sequence Diagrams

Ownership Report

Name	Work done
Yehia Hassan Abdelmoaty	Class Diagram and Descriptions
Noha Mohamed Ahmed	State diagram
Jana Ahmed Farhat Hassan	Architecture Diagram and Descriptions
Hana Khaled Abdelhamed	Sequence Diagram and Descriptions